

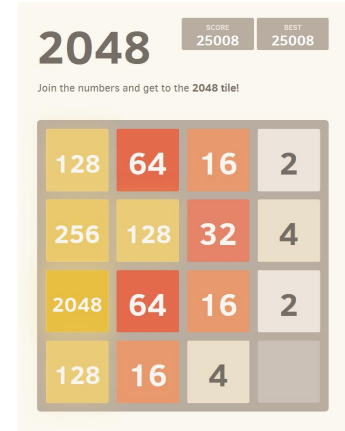
Designing an AI Agent for Game 2048

Yu Ji, Chao Wang, Shuying Zhang
Stanford University

BACKGROUND

- 2048 is a puzzle game on mobile devices that went viral in 2015. The game rule is very simple: join the numbers and get to 2048 tile. However, the game is notoriously hard to win and insanely addictive. Only 1 percent of all games have been able to get that far. According to the game designer Gabriele Cirulli, “assuming the average 2048 game lasts 8 minutes (guess), 521 years have already been spent playing the game.” In order to be part of the “one percent” that beat the game, we proposed an AI agent to play the game.

- Deep Q-learning has shown promising result in games such as Atari [1] and FlappyBird. Deep Q-learning is based on Q-learning and learn the cost of minimum path Q by leveraging the power of neural network. Unlike vanilla Q-learning, which attempts to learn the entire Q-table, deep Q-learning learns an estimate of the Q function that spells out an action given any state.



- Expectimax approach consists of a random agent and an maximizing agent. In this game, the game is a random agent, which adds a new tile at a random empty cell after each move; the AI agent is a maximizing agent, which tries to maximize the score. An evaluation function evaluates the maximum future reward by leveraging domain knowledge about the game.

PROBLEM STATEMENT

We are aimed to design an AI agent to get 2048 tile in the 2048 game.

DATASET PREPROCESSING

Input Data for Neural Network

The grid is a 4x4 matrix, which is normalized by takes log of 2. Then it is expanded to a binary map represented as a 4x4x16 matrix.

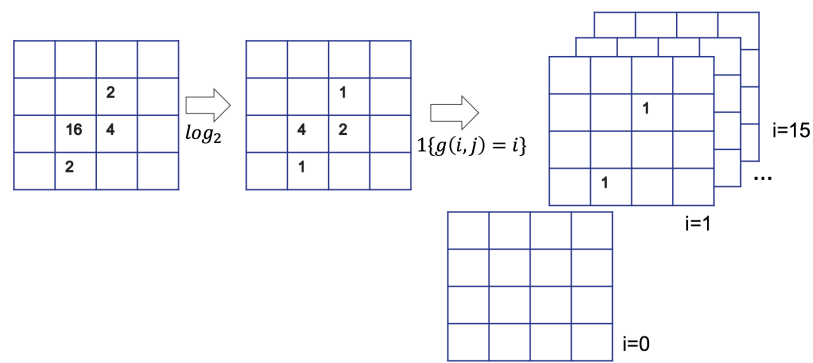


Figure 1. Input data for neural network. Each 4x4 grid is converted to a 4x4x16 matrix.

Training Batch

For each train step, we transform the resulting grid from the last step into 8 grids by rotation. Since each grid is positional invariant to each other, they have the same Q, which is the label for the training data. And these 8 grids consists a batch and feeds to the network to generate the next action.

MATERIALS AND METHODS

Baseline

We used the highest immediate reward strategy as the baseline. This strategy takes the action that gives highest score for one step.

Expectimax

There are two agents in this game: the game (opponent) as a random agent, which adds a new tile at a random empty cell after each move; the AI agent as a maximizing agent, which tries to maximize the score. The recurrence is defined as follows:

$$V_{max,opp}(s, d) = \begin{cases} \frac{Utility(s)}{Eval(s)} & IsEnd(s) \\ \max_{a \in Actions(s)} V_{max,opp}(Succ(s, a), d) & d = 0 \\ \sum_{a \in Actions(s)} \pi_{opp}(s, a) V_{max,opp}(Succ(s, a), d - 1) & Player(s) = agent \\ & Player(s) = opp \end{cases}$$

Evaluation function

We linearly combined five features and the weight as the evaluation function: maximum number in the grid A , number of free tiles f , number of adjacent cells with same number a , sum of edge cells e , sum of absolute differences between adjacent cells d :

$$Eval(s) = A * w1 + f * w2 + a * w3 + e * w4 + d * w5$$

Deep Q-learning

Q-learning outsmarts highest immediate reward strategy by not only taking into account the immediate rewards but also future rewards to maximum total future reward. Since the same action for the same state leads to the same result, we use discounted future reward γ equals to 1 to solve this problem. We define a function $Q(s, a)$ representing the maximum discounted future reward when we perform action a in state s , and continue optimally from that point on.

$$Q(s_t, a_t) = \max R_{t+1}$$

We iteratively approximate the Q-function using the Bellman equation:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

The state is defined as (g, score) where g is the current grid; score is the current game score. The four possible actions are up, down, left and right.

Learning Q-function with Neural Network

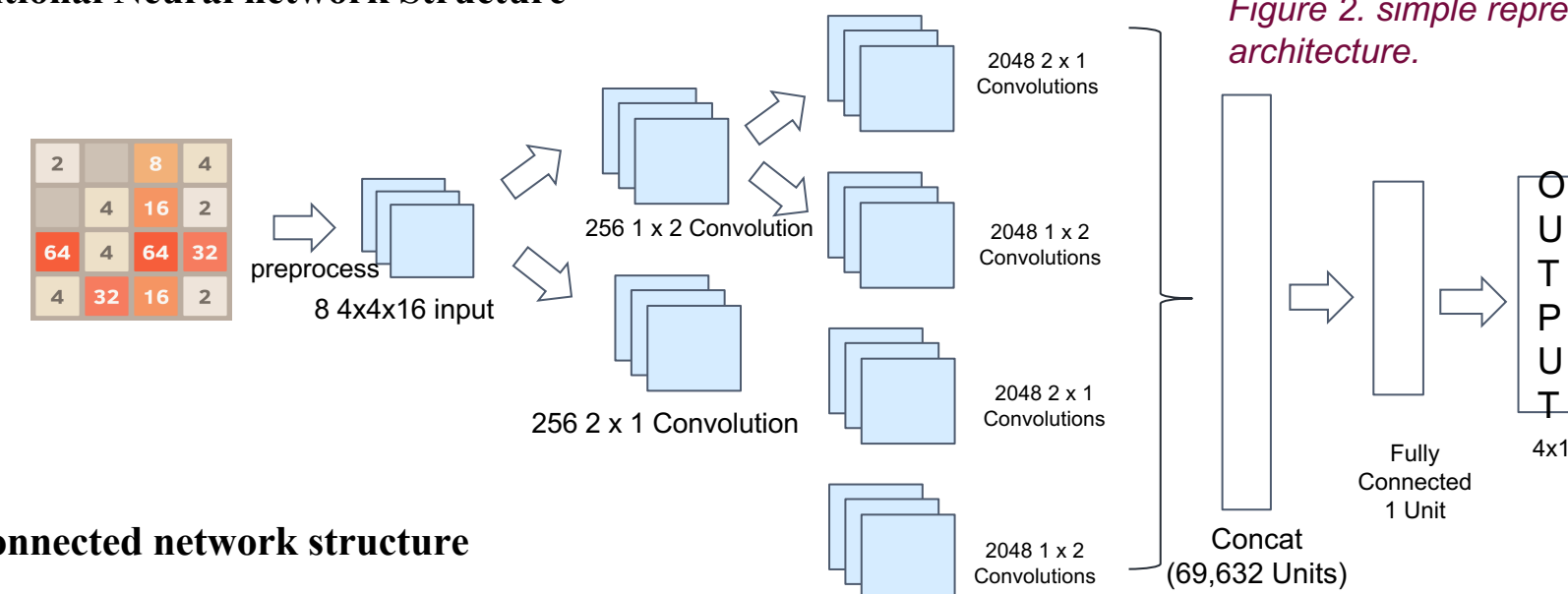
For each step of the game, we transform the current game state into one training batch and feed to the neural network to output a 1x4 vector representing the Q values of the 4 possible actions (Figure 2). The next action is the action that gives the maximum Q.

We use the square loss between target $r + \max_{a'} Q(s', a')$ and the prediction $Q(s, a)$:

$$L = \frac{1}{2} [r + \max_{a'} Q(s', a') - Q(s, a)]^2$$

The initial learning rate is 0.001. Adam optimization was used.

Convolutional Neural network Structure



Fully Connected network structure

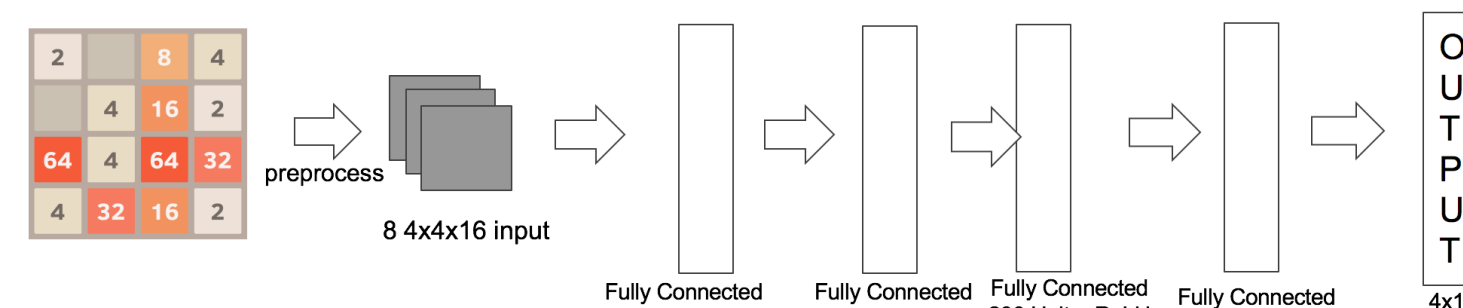


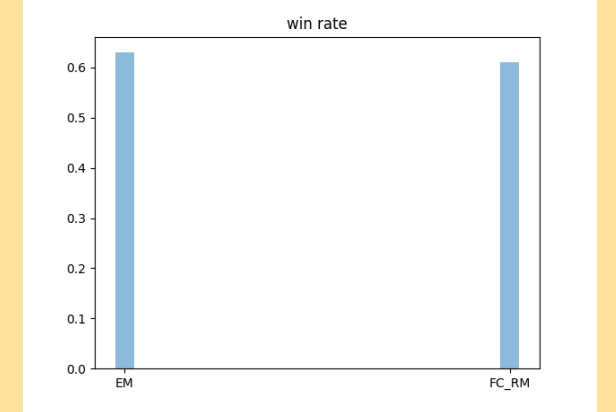
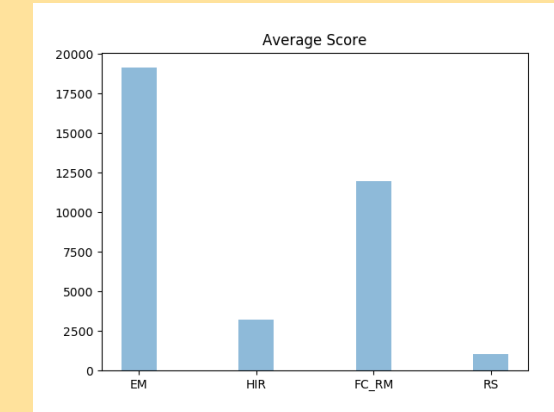
Figure 2. simple representation of Deep Q-network architecture.

Figure 3. Convnet architecture

Figure 4. Fully-connected architecture

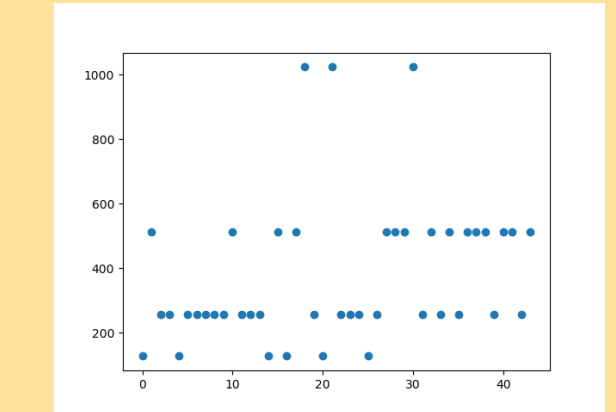
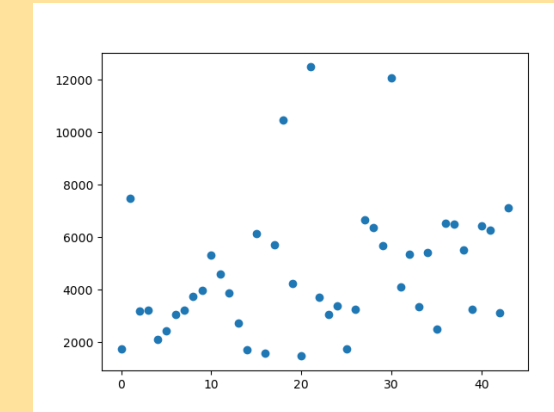
RESULTS

1. The average scores and winning rates are shown as follows.



Expectimax has the largest average score and highest winning rate 63%.

2. Training scores and maximum number of CNN



CONCLUSIONS

- We have shown that our AI agent can learn the game and outperform baseline strategy and win the game.
- Expectimax shows the best result so far.
- Fully connected and convolutional neural network can also win the game, but time-consuming to train.

FUTURE WORK

- We will keep tuning our neural network. Our current performance is limited by the computational resources. Training 30 steps of the convnet with a CPU took around 5 hours. We will try faster computational tool such as a GPU for training in the future.
- We will also try different regularization methods such as batch normalization and dropout.

REFERENCES

- Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." *arXiv preprint arXiv:1312.5602* (2013).